

Tailoring DINOv3 for Semantic Segmentation

An implementation walkthrough of the linear-head and Mask2Former heads released in
[facebookresearch/dinov3](https://github.com/facebookresearch/dinov3)

Coursework report — DINOv3 study project

Abstract

This report describes how the official DINOv3 codebase (<https://github.com/facebookresearch/dinov3>) adapts a frozen DINOv3 ViT backbone to semantic segmentation. Two adaptations are released by Meta and are covered here in detail: a minimal **linear head** (a single 1×1 convolution on patch features), trained end-to-end on ADE20K, and a heavyweight **Mask2Former (M2F)** head fed by a **DINOv3-Adapter** that turns the single-scale ViT into a multi-scale feature pyramid. The first is fully trainable from the released code; the second is shipped as inference-only with a pretrained checkpoint that achieves the headline 55.9 mIoU on ADE20K reported in Meta’s model card. The report is grounded in the source code: every claim cites a file and line range so a reader can verify it. We do not reproduce Meta’s source code in this document, only describe and reference it.

Contents

1	What semantic segmentation needs from a backbone	2
2	Where the segmentation code lives	2
3	The frozen-backbone interface	3
4	The linear segmentation head	3
4.1	Architecture	3
4.2	Training recipe	4
4.3	Sliding-window inference	4
4.4	Loss	5
5	The Mask2Former head	5
5.1	Top-level wiring	5
5.2	Multi-scale deformable attention	6
5.3	Inference-time output	6
5.4	What is and isn’t shipped	6
6	The DINOv3-Adapter	7
6.1	Approach in plain terms	7
6.2	Why this works at all	7
6.3	Construction details	7
7	Training the linear head: end-to-end flow	8
8	Reduced single-GPU recipe	8

9	Custom-image inference	9
10	Installing it: the practical bits	9
11	Expected results	10
12	Why DINOv3 features are well-suited to segmentation	10
13	File-by-file map	10

1 What semantic segmentation needs from a backbone

Semantic segmentation requires a per-pixel class prediction. A ViT backbone naturally produces per-*patch* features — for a 512×512 input with patch size 16, a 32×32 grid of 1024 feature vectors. To get a per-pixel prediction at resolution $H \times W$, the segmentation pipeline must:

1. extract patch features from the backbone, possibly from several depths in the network;
2. combine and project them into class logits per patch;
3. upsample the patch-level logits to pixel resolution.

The two heads in DINOv3 differ in how much work they do at each step. The *linear* head is intentionally trivial (concatenate features along the channel axis, BatchNorm, 1×1 Conv, bilinear upsample); its purpose is to demonstrate that the backbone features are already class-discriminative *without* a heavy decoder. The *M2F* head, by contrast, treats segmentation as mask classification: it learns a fixed set of object queries and predicts masks per query, which means it needs a multi-scale feature pyramid the ViT does not natively provide — hence the *DINOv3-Adapter*. We unpack both in turn.

2 Where the segmentation code lives

Where to look in the official DINOv3 repository

`dinov3/eval/segmentation/` (the whole subtree).
Plus `dinov3/hub/segmentors.py` for the prebuilt M2F segmentor loader, and `dinov3/eval/utils.py:ModelWithIntermediateLayers`.

A summary of the directory:

`models/__init__.py`

the dispatcher `build_segmentation_decoder()`: chooses linear vs. M2F, wires the backbone to the chosen head, and returns a single `FeatureDecoder` module that exposes a uniform `forward()/predict()` interface to the trainer and the evaluator.

`models/heads/`

the heads themselves — `linear_head.py` and `mask2former_head.py` (plus its `pixel_decoder.py` and `mask2former_transformer_decoder.py`).

`models/backbone/dinov3_adapter.py`

the multi-scale adapter used only by M2F.

`models/utils/`

support layers shared by M2F: deformable attention (custom CUDA op), positional encodings, helper transformer blocks, batch-norm wrappers.

`train.py`, `eval.py`, `run.py`

train and eval loops; `run.py` is the entry point both invoke.

`inference.py`

the sliding-window inference utility used at evaluation time and by our custom-image script.

`loss.py`

cross-entropy and Dice losses with class weighting and ignore-index support.

`metrics.py`

mIoU computation.

`transforms.py`
paired image+mask data augmentations.

`configs/config-ade20k-linear-training.yaml`
the training recipe for the linear head.

`configs/config-ade20k-m2f-inference.yaml`
the *inference-only* recipe for M2F.

3 The frozen-backbone interface

Where to look in the official DINOv3 repository

`dinov3/eval/utils.py` (the class `ModelWithIntermediateLayers`),
`dinov3/models/vision_transformer.py:get_intermediate_layers`.

Both heads consume the same backbone interface: a list of patch-feature maps extracted from selected transformer blocks. The plumbing is:

1. The DINOv3 ViT exposes `get_intermediate_layers(x, n=..., reshape=True, return_class_token=...)` which returns, for each requested block index, a tensor of shape $(B, d, H/P, W/P)$ if `reshape=True` (so it is already a 2D feature map), and optionally the corresponding (B, d) CLS token.
2. `ModelWithIntermediateLayers` wraps that in a callable module and adds an autocast context. It also calls `requires_grad_(False)` on the wrapped backbone, freezing it for the segmentation experiments.
3. Which blocks to extract is controlled by the `BackboneLayersSet` enum in `eval/segmentation/models/__init__.py`:
 - **LAST**: only the last block. Used by the linear head.
 - **FOUR_LAST**: the last four blocks.
 - **FOUR_EVEN_INTERVALS**: four blocks evenly spaced. For ViT-L/16 (24 blocks) the indices are `[4, 11, 17, 23]` – this is what the M2F config uses.

In short: the head receives a list of one or four 2D feature maps (and optionally CLS tokens) at the patch-grid resolution. The head's job is to turn those into class logits at pixel resolution.

4 The linear segmentation head

Where to look in the official DINOv3 repository

`dinov3/eval/segmentation/models/heads/linear_head.py`,
`configs/config-ade20k-linear-training.yaml`,
`eval/segmentation/train.py` (training loop),
`eval/segmentation/loss.py`, `eval/segmentation/metrics.py`.

4.1 Architecture

The linear head is approximately 90 lines of code and is essentially a single 1×1 convolution. Its construction takes:

- `in_channels`: a list of per-feature-map dimensions. For the default **LAST** configuration with a ViT-L backbone the list is just `[1024]`; for **FOUR_EVEN_INTERVALS** it would be `[1024, 1024, 1024, 1024]`.

- **n_output_channels**: number of classes (150 for ADE20K, plus an ignore index handled by the loss).
- **use_batchnorm** (default **true**): a `nn.SyncBatchNorm` on the concatenated channels.
- **use_cls_token** (default **false** for the ADE20K config): if true, the CLS token is broadcast spatially and concatenated with the patch tokens, doubling the channel count.
- **dropout** (default 0.1): `nn.Dropout2d` applied before the 1×1 convolution.

The forward pass is:

1. If **use_cls_token**, broadcast each block’s CLS token $(B, d) \rightarrow (B, d, h, w)$ and concatenate with that block’s patch map along the channel axis.
2. Bilinearly upsample every block’s feature map to the spatial resolution of the first block (this is a no-op when only the last block is extracted) and concatenate them along the channel axis. The result is a single dense map $(B, \sum d_i, h, w)$.
3. `Dropout2d` $\rightarrow$ `BatchNorm` $\rightarrow$ `Conv2d1×1`.
4. For prediction, bilinearly upsample the (B, K, h, w) logits to the requested target size (typically the input image’s spatial resolution).

The intent of this head is pedagogical and methodological: it shows that DINOv3’s patch features are linearly separable into the 150 ADE20K classes to a useful degree without a heavy decoder. The head adds only $K \cdot \sum d_i$ parameters (e.g. $150 \cdot 1024 = 153,600$ for ViT-L/16 LAST), plus the BatchNorm scale/shift – against the millions in the backbone.

4.2 Training recipe

Reading `configs/config-ade20k-linear-training.yaml` top-to-bottom gives the exact recipe:

- **Hardware**: 8 GPUs, batch size 2 each (effective batch 16).
- **Iterations**: 40 000 total.
- **Optimizer**: AdamW, $\beta_1=0.9$, $\beta_2=0.999$, weight decay 10^{-3} , learning rate 10^{-3} . Gradient clipping disabled ("`inf`").
- **Schedule**: `WarmupOneCycleLR`: linear warmup over 1500 iterations from 10^{-9} up to peak 10^{-3} , then a single cosine descent. `pct_start` is 0 which makes the “warm-up” part part of the descent path; the effective peak occurs immediately after warm-up.
- **Loss**: Cross-entropy weight 1.0, Dice weight 0.0. (Dice is implemented in `loss.py` but disabled in this config.)
- **Class handling**: `reduce_zero_label: true` maps ADE20K’s class 0 (“unlabelled”) to the ignore index, so the head learns 150 classes only.
- **Augmentations**: random scale ratio in $[0.5, 2.0]$, random crop to 512×512 , random horizontal flip with probability 0.5. The whole transform is implemented as a paired operation on (image, mask) so the mask follows every geometric change.
- **Eval**: sliding-window inference at 512×512 with stride 341 ($\approx \frac{2}{3}$ overlap), evaluated every 5000 iterations. No test-time augmentation. Metric: mean IoU over the 150 valid classes.

4.3 Sliding-window inference

Where to look in the official DINOv3 repository

```
dinov3/eval/segmentation/inference.py:slide_inference.
```

The eval procedure is sliding-window with overlap. Given an input $(1, 3, H, W)$, a crop size $h_c \times w_c$, and a stride $h_s \times w_s$, the function:

1. Computes the grid of crops needed to tile the image, where the last column/row is shifted in to avoid going out of bounds.
2. For each crop, runs the segmentation model and accumulates its prediction into a (CPU, padded) buffer of shape $(1, K, H, W)$.
3. Maintains a count map of how many crops have contributed to each pixel.
4. Returns the buffer divided by the count map, i.e. the average prediction per pixel.

At training time, the loss is computed on 512×512 random crops, so the model never actually sees the full image; the sliding-window protocol at eval time is what converts that into a coherent full-image prediction. The overlap of $\frac{2}{3}$ is generous and reduces edge artefacts.

For our purposes the key fact is that `eval/segmentation/inference.py:make_inference` wraps `slide_inference` with: an optional rescale-to-target step, an optional horizontal-flip TTA, and post-processing for the M2F head’s mask-classification output. Our custom-image script (Section 9) calls into `make_inference` directly.

4.4 Loss

Where to look in the official DINOv3 repository

`dinov3/eval/segmentation/loss.py`.

`loss.py` provides two pieces: a weighted-cross-entropy loss with ignore index, and a soft-Dice loss (mostly disabled in the released ADE20K config but available). Both follow the convention of returning a per-element loss that can be combined with a weight tensor. The pattern `weighted_loss` (a decorator factory) wraps a raw element-wise loss function with weight, reduction, and avg-factor handling. In the ADE20K linear config only the cross-entropy is active.

5 The Mask2Former head

Where to look in the official DINOv3 repository

`dinov3/eval/segmentation/models/heads/mask2former_head.py`,
`.../heads/mask2former_transformer_decoder.py`,
`.../heads/pixel_decoder.py`,
`.../models/utils/` (deformable attention, position encoding, etc.),
`.../models/backbone/dinov3_adapter.py` (next section),
`configs/config-ade20k-m2f-inference.yaml`.

Mask2Former (Cheng et al.) reformulates segmentation as mask classification. Instead of predicting a class per pixel, the model predicts a fixed set of N object queries; each query produces a binary mask plus a class label. The final per-pixel class is obtained by summing $\text{mask}_i \cdot p(\text{class}|i)$ across queries. This unifies semantic, instance, and panoptic segmentation under one architecture and typically gives substantially higher mIoU than per-pixel classification.

5.1 Top-level wiring

The class `Mask2FormerHead` is constructed inside `build_segmentation_decoder()` when `decoder_type="m2f"`. It expects an `input_shape` dict describing the four feature levels produced by the DINOv3-Adapter (Section 6); for ViT-L the keys are "1"- "4" and the spatial strides decrease from $P/2$ at the deepest level to $4P$ at the shallowest, where P is the patch size.

The two main sub-modules are:

Pixel decoder.

Implemented in `pixel_decoder.py`. It is a FPN-with-deformable-attention: the multi-scale features are first pushed through a small Transformer encoder that uses *multi-scale deformable attention* (MSDeformAttn), then progressively upsampled and laterally combined to produce a single high-resolution mask feature map (typically at stride 4).

Transformer decoder.

Implemented in `mask2former_transformer_decoder.py`. It takes a fixed set of learnable query embeddings and refines them through several layers of *masked cross-attention* (each query attends only to the predicted-mask region of the encoder features), self-attention, and FFN. Each layer outputs intermediate masks + class logits used for deep supervision (during training) and layer-wise inference.

5.2 Multi-scale deformable attention

Where to look in the official DINOv3 repository

`models/utils/ms_deform_attn.py`, `models/utils/ops/` (the CUDA extension).

Standard cross-attention on dense image features at multiple scales is quadratic in the number of pixels. Deformable attention replaces it with a sampling operation: each query predicts a small set of (offset, attention weight) pairs; the value vectors at those offsets are bilinearly sampled and combined. The result is linear in the number of queries and effectively constant per scale. The CUDA op MSDeformAttn implements this sampling efficiently. Compiling this extension is the single most fragile step of installing M2F (see Section 10).

5.3 Inference-time output

The decoder produces, at the last layer, a tensor of *mask logits* of shape (B, N, h, w) (per-query masks, where N is the number of queries) and a tensor of *class logits* of shape $(B, N, K+1)$ (per-query class scores; the extra slot is a “no-object” label). The inference recipe in `inference.py:make_inference` is:

1. Softmax the class logits over $K+1$, drop the last entry (the no-object slot).
2. Sigmoid the mask logits.
3. Combine via the einsum $\text{out}_{b,c,h,w} = \sum_q p_{b,q,c} \cdot m_{b,q,h,w}$, which gives a per-pixel class score map of shape (B, K, h, w) .
4. Argmax over K to get the per-pixel class.

The released ADE20K M2F head also uses test-time augmentation: scales $[0.9, 0.95, 1.0, 1.05, 1.1]$ averaged, horizontal flips averaged, sliding-window crops 896×896 with stride 596. This is what gets the model to 55.9 mIoU on ADE20K val (per Meta’s model card).

5.4 What is and isn’t shipped

The M2F training-time machinery (Hungarian matcher, per-mask BCE+Dice criterion, deep-supervision losses) is present in `mask2former_head.py` for completeness, but the only released *config* is the inference one. Meta releases a pretrained M2F head checkpoint (loadable via `dinov3.hub.segmentors.dinov3_vit7b16_ms`) and the recipe to evaluate it. There is no shipped training config for M2F, so re-training it on a different dataset would require writing one from scratch. For this project we use M2F as inference-only.

6 The DINOv3-Adapter

Where to look in the official DINOv3 repository

`dinov3/eval/segmentation/models/backbone/dinov3_adapter.py`.

This is the part of the M2F path that is novel to DINOv3 (the rest of M2F is essentially the original Cheng-et-al. recipe). Mask2Former needs a multi-scale feature pyramid; DINOv3 only produces single-scale patch features (one feature per 16×16 patch). The adapter manufactures the missing scales.

6.1 Approach in plain terms

Given the ViT’s intermediate features at four equally-spaced blocks (the `FOUR_EVEN_INTERVALS` setting), the adapter:

1. Up-samples the shallowest extracted block twice with deconvolutions (each $\times 2$) to produce a feature map at stride $P/4$ ($= 4$ for $P=16$). This becomes feature level "1".
2. Up-samples the next block once to produce a stride- $P/2$ ($= 8$) feature map. This becomes level "2".
3. Keeps the third extracted block unchanged at stride P ($= 16$). This is level "3".
4. Down-samples the deepest extracted block once with a strided convolution to produce a stride- $2P$ ($= 32$) feature map. This is level "4".

The four resulting maps form a feature pyramid spanning strides 4, 8, 16, 32, exactly what M2F’s pixel decoder expects. All four maps share the same channel dimension as the ViT’s embed dim (e.g. 1024 for ViT-L, 4096 for ViT-7B), so the adapter introduces only a few small deconv/conv layers in addition to the frozen backbone.

6.2 Why this works at all

Treating different layers of the ViT as different “scales” is a trick that has been shown to work because deeper ViT layers have larger effective receptive fields and more semantic features, while shallower layers retain more local detail. Combined with deconvolutional up-sampling for the shallow levels and strided convolution for the deep level, the adapter approximates the multi-scale behaviour of a hierarchical convnet. The fact that DINOv3 features are already spatially coherent (thanks to the Gram-anchoring stage during pretraining) means the upsampled maps actually retain useful local structure rather than smoothing into uniform blobs. This is the contribution of the DINOv3-Adapter: a minimal way to inject the multi-scale assumption back in.

6.3 Construction details

`DINOv3_Adapter` takes the backbone module and a list `interaction_indexes` (the four block indices to extract). Its forward pass calls into the backbone’s `get_intermediate_layers(reshape=True)` once, getting a tuple of four 2D maps, then dispatches each through the appropriate deconv/identity/conv head and returns a dict keyed by level. This dict is fed to the M2F pixel decoder.

7 Training the linear head: end-to-end flow

Where to look in the official DINOv3 repository

```
eval/segmentation/run.py,                      eval/segmentation/train.py,  
eval/segmentation/transforms.py, data/datasets/ade20k.py.
```

A complete training run, from input to backward pass, is:

1. **Dataset.** ADE20K (in `dinov3/data/datasets/ade20k.py`) reads ADEChallengeData2016/ via `torchvision` primitives. Each sample is a (PIL image, PIL mask) pair; the mask uses ADE20K’s index encoding (0 = unlabelled, 1...150 = classes).
2. **Paired transforms.** `transforms.py` defines a `Compose`-style class where every operation acts on (image, mask) together: `RandomResize` (scale ratio from a [0.5, 2.0] uniform), `RandomCrop` to 512×512, `RandomHorizontalFlip` with $p=0.5$, then `Normalize` (image only) with the LVD-1689M mean/std.
3. **Forward.** `model.forward()` is the wrapped `FeatureDecoder` chain: backbone (frozen, in inference_mode) → extract intermediate layers → `LinearHead.forward()` → raw (B, K, h, w) logits. The training loop passes those logits and the resized ground-truth mask to the loss.
4. **Loss.** Cross-entropy with `ignore_index=255` (the value the dataloader uses to mark “ignore”). `reduce_zero_label` in the config makes the dataloader subtract 1 from every valid class so that 0 becomes 255 (ignore) and class IDs are now 0...149.
5. **Backward.** Standard `loss.backward()` → `optimizer.step()`. Because the backbone is frozen, gradients flow only into the linear head’s BatchNorm and 1×1 Conv. This is the reason the head trains so quickly despite the large model.
6. **Validation.** Every 5000 iterations the eval routine runs sliding-window inference on the ADE20K validation set (Section 4.3) and reports mIoU. The metric is accumulated across the full val set in a confusion matrix (see `metrics.py`).

8 Reduced single-GPU recipe

The released config assumes 8 GPUs. We supply `configs/config-ade20k-linear-training-1gpu.yaml` as a single-GPU variant. We changed *only* hardware-related fields:

- `n_gpus: 1, bs: 1`: effective batch 1 instead of 16. With a frozen backbone and a tiny head this is OK in practice (BatchNorm sees only one sample per step, but the head’s BatchNorm is a single-channel per-class scaling that can be replaced with a no-op or kept as-is for evaluation).
- `transforms.train.img_size: 512`: same as the multi-GPU config; can be reduced to 448 if the chosen backbone is too large for the GPU.
- Crop size and stride at eval are unchanged (512 / 341).

Algorithmic settings – optimizer, scheduler, loss weights, head structure, `reduce_zero_label`, etc. – are preserved exactly. Whether the 1-GPU run actually converges to mIoU within a few points of the 8-GPU run depends on the smaller effective batch size; for our class demo we are comfortable with the qualitative result and the partial-training checkpoint as a teaching artefact.

9 Custom-image inference

The repository accompanying this report contains one script that does not exist in Meta’s repo: `scripts/segment_image.py`. Its only role is to invoke the existing pieces correctly on a single user image.

The pipeline it implements:

1. Load the image with PIL, normalise with the LVD-1689M mean/std (since the released backbones are pretrained on web images), optionally resize the short side to a target size.
2. Build the chosen backbone via Meta’s hub factory (`dinov3.hub.backbones.dinov3_vits16`, etc.), passing the local checkpoint path as `weights=`.
3. Build the chosen decoder via `dinov3.eval.segmentation.models.build_segmentation_decoder` with `decoder_type={linear,m2f}`; the function returns a `FeatureDecoder` that already wraps the backbone + head.
4. Load the decoder checkpoint with `strict=False`, mirroring the way Meta’s `dinov3/hub/segmentors.py` loads the released M2F head (the backbone keys are skipped because the backbone was loaded separately).
5. Call `dinov3.eval.segmentation.inference.make_inference` with `inference_mode="slide"` and the head-appropriate crop / stride: (512, 341) for the linear head and (896, 596) for M2F, matching Meta’s eval configs.
6. Convert the output logits to a class map via `argmax`, colour-code the classes, and overlay on the original image.

The script is roughly 300 lines, of which the modelling-relevant parts are simple wrappers around the four `imports` above. The colour palette and the overlay blending are routine numpy. We deliberately do not bundle ADE20K’s official palette file with the repository; instead we generate a deterministic palette via HSV with a fixed seed so the visualisations are reproducible.

10 Installing it: the practical bits

The project’s `README.md` has the full walkthrough. Key practical points:

Cloning vs. vendoring. We *do not* include Meta’s source in this repository. Instead, the user clones `facebookresearch/dinov3` alongside, installs it with `pip install -e .`, and our scripts import it as a normal package. This keeps the licence story clear and the repository small.

The MSDeformAttn extension. Mask2Former requires a custom CUDA op compiled at install time (`python setup.py build_ext -inplace` in `dinov3/eval/segmentation/models/utils/ops/`). The most common failure modes are: (a) CUDA toolkit version mismatched with the PyTorch build, (b) `nvcc` not on `PATH`, (c) `TORCH_CUDA_ARCH_LIST` unset on machines whose driver misreports the device capability. The op is needed only for M2F; the linear head runs without it.

Pretrained weights. Three to four checkpoints in total: a backbone (any of S/B/L/7B), the released ADE20K M2F head, and any linear head that you train yourself. The backbones are gated behind a Meta download form. The data (ADE20K) is gated behind the MIT release form. Neither is redistributable.

11 Expected results

The headline numbers, taken from Meta’s `MODEL_CARD.md`, are:

Model	ADE20K mIoU	Notes
DINOv3 ViT-L/16 + linear head	54.9	frozen backbone, 1×1 Conv head
DINOv3 ViT-7B/16 + linear head	55.9	frozen backbone, 1×1 Conv head
DINOv3 ViT-7B/16 + M2F head	55.9	released checkpoint, TTA at 5 scales

That the 7B linear head matches the 7B M2F head on this metric is, by itself, a striking result – it suggests that for ADE20K the bottleneck is the backbone, not the decoder. M2F retains its advantage on instance- and panoptic-segmentation metrics that the linear head cannot reach by construction (the linear head always assigns the same class to all pixels in a region; it has no notion of separate instances).

For our class project we run two demonstrations:

1. Train the linear head on ADE20K (single-GPU recipe), report train/val curves, and evaluate at the iteration we can afford to reach (typically a few thousand iterations on a single consumer GPU). The checkpoint we obtain will not match the 54.9 mIoU figure above but should clearly demonstrate the training behaviour and produce sensible qualitative outputs on ADE20K val and on custom images.
2. Run M2F inference at full quality on ADE20K val and on custom images, reproducing the qualitative results from the model card.

12 Why DINOv3 features are well-suited to segmentation

Three properties of the DINOv3 backbone matter directly for segmentation quality:

Patch features are spatially coherent. The Gram-anchoring training stage explicitly preserves the pairwise patch-similarity structure across long training, which is the property a segmentation head exploits when grouping nearby patches into the same class.

Register tokens absorb high-norm artefacts. In ViTs trained without registers, a small number of patch tokens become high-norm “sinks” that hold global information; those positions appear as bright artefacts in segmentation outputs. DINOv3 includes 4 register tokens explicitly to absorb this role, leaving patch tokens spatially clean and class-discriminative.

High-resolution adaptation. The third pretraining stage trains the model at multiple resolutions including 768, so applying it at ADE20K’s typical 512–896 inputs is in-distribution. With learnable absolute positional embeddings (DINOv2) this would have required interpolation; with axial RoPE (DINOv3) it is native.

These three together mean that a 1×1 convolution on top of frozen DINOv3 features achieves a result that, three years ago, would have required a heavy task-specific decoder.

13 File-by-file map

File in dinov3/	Role
<code>models/vision_transformer.py</code>	ViT backbone with the <code>get_intermediate_layers</code>
<code>eval/utils.py</code>	<code>ModelWithIntermediateLayers</code> (frozen-backbone)
<code>eval/segmentation/models/__init__.py</code>	dispatcher: linear vs M2F
<code>eval/segmentation/models/heads/linear_head.py</code>	linear segmentation head
<code>eval/segmentation/models/heads/mask2former_head.py</code>	M2F top-level head
<code>eval/segmentation/models/heads/mask2former_transformer_decoder.py</code>	M2F transformer decoder
<code>eval/segmentation/models/heads/pixel_decoder.py</code>	M2F pixel decoder (FPN + MSDeformAttn)
<code>eval/segmentation/models/backbone/dinov3_adapter.py</code>	multi-scale adapter for M2F
<code>eval/segmentation/models/utils/ms_deform_attn.py</code>	deformable attention layer
<code>eval/segmentation/models/utils/ops/</code>	CUDA extension for MSDeformAttn
<code>eval/segmentation/inference.py</code>	sliding-window + TTA inference
<code>eval/segmentation/train.py, eval.py, run.py</code>	training and evaluation loops
<code>eval/segmentation/loss.py, metrics.py</code>	cross-entropy/Dice losses, mIoU
<code>eval/segmentation/transforms.py</code>	paired (image, mask) augmentations
<code>eval/segmentation/configs/</code>	ADE20K linear-training and M2F-inference YAM
<code>hub/segmentors.py</code>	prebuilt M2F segmentor with weight downloading

This concludes the report. The companion repository contains the README, the single-GPU config, and the custom-image inference script that wires all of the above into a runnable demo on user-supplied images.